Actividad N°2 – AWS IoT – Node-RED

Grupo 03

Integrantes:

- Daltroy Mateo
- Lima Francisco
- Lamiral Matheo

Contexto inicial

El objetivo de la actividad es simular el control de la iluminación de un ambiente con dos instancias que se comunican por **MQTT** usando como bróker **AWS IoT**. La **Instancia A** funciona como *control remoto* (botones ON/OFF, estado de la luminaria y función TIMER de 5 s) y la **Instancia B** como la *luminaria*, que muestra el estado del LED y del TIMER. Las dos aplicaciones se desarrollaron con **Node-RED** y el paquete `@flowfuse/node-red-dashboard`. Todo el trabajo se hizo en la región **Ohio (us-east-2)**.

El despliegue de las instancias t3.micro con Debian 13 y la instalación de Node-RED se realizaron de la misma forma que en las prácticas anteriores, por lo que no se detallan en este informe.

1) Instalación del nodo @flowfuse/node-red-dashboard

Desde el editor de Node-RED, en **User Settings** → **Palette** → **Install**, buscamos el paquete `@flowfuse/node-red-dashboard` (versión 1.32.0) y confirmamos la instalación con **Install**. Este paquete agrega los nodos de interfaz (botones, switch, texto, gauge) que usamos para armar los dashboards.

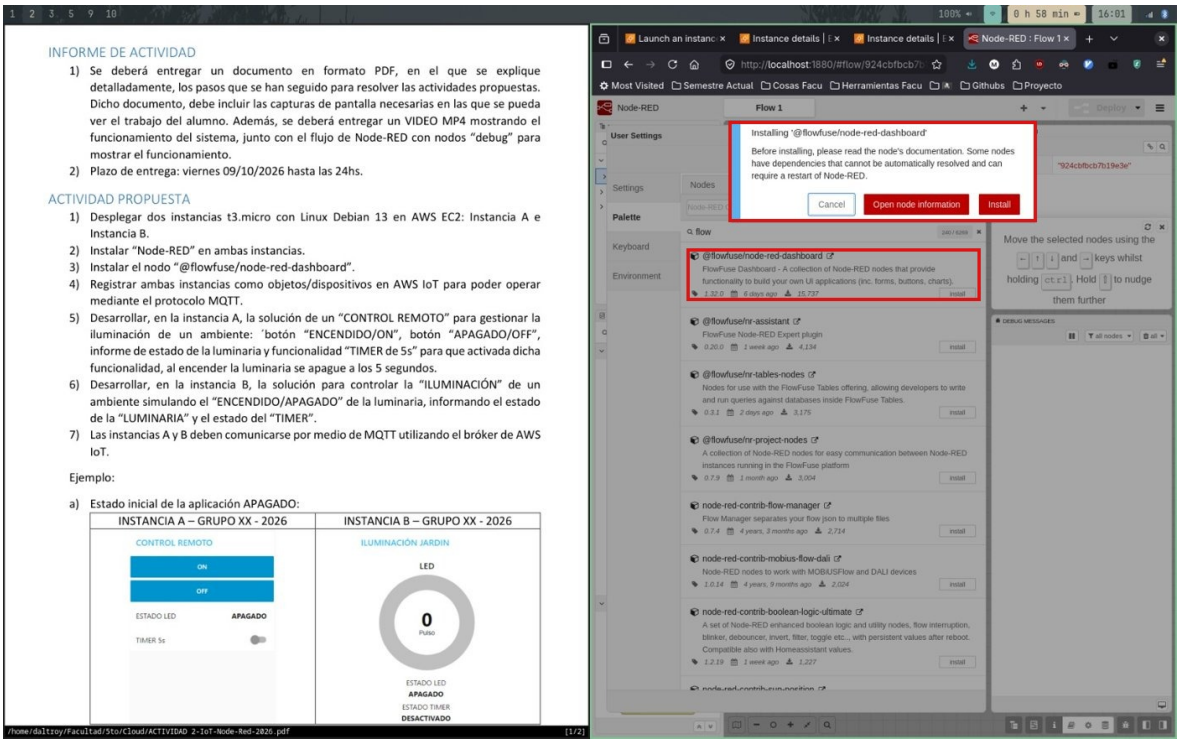


Figura 1: instalación de @flowfuse/node-red-dashboard desde la paleta.

2) Registro en AWS IoT

Creación de los objetos (things)

En la consola de **AWS IoT** → **Connect one device** registramos el dispositivo `sensor-G03`. Al terminar el asistente, AWS IoT confirma la creación del objeto y genera el **kit de conexión**. También registramos el objeto `G03-Iluminacion`; ambos quedan listados en **Manage** → **All devices** → **Things**.

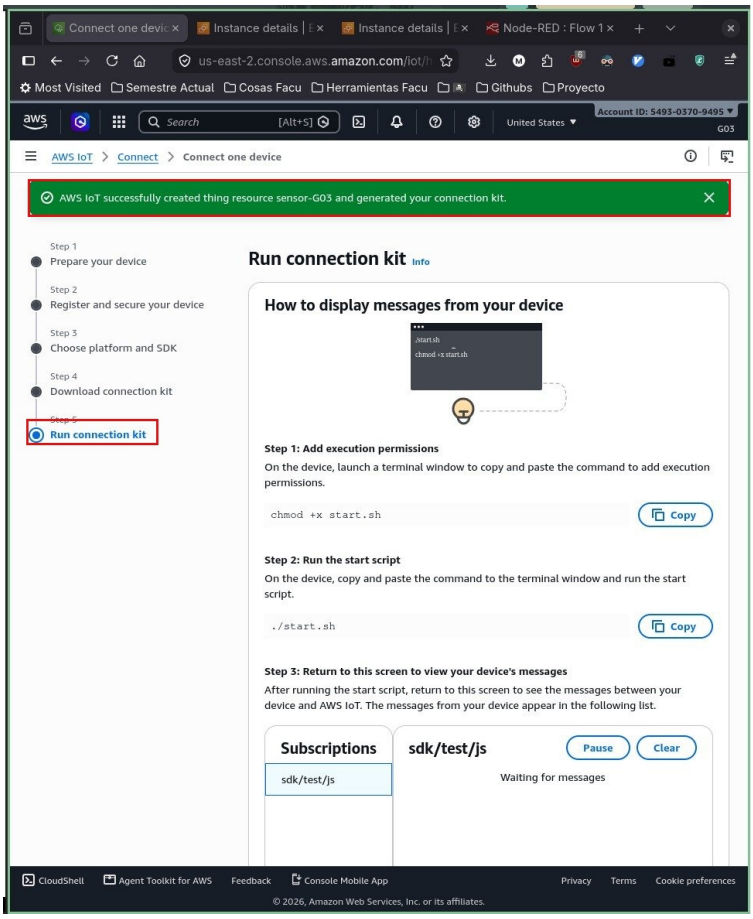


Figura 2: AWS IoT confirma la creación de sensor-G03 y genera el kit de conexión.

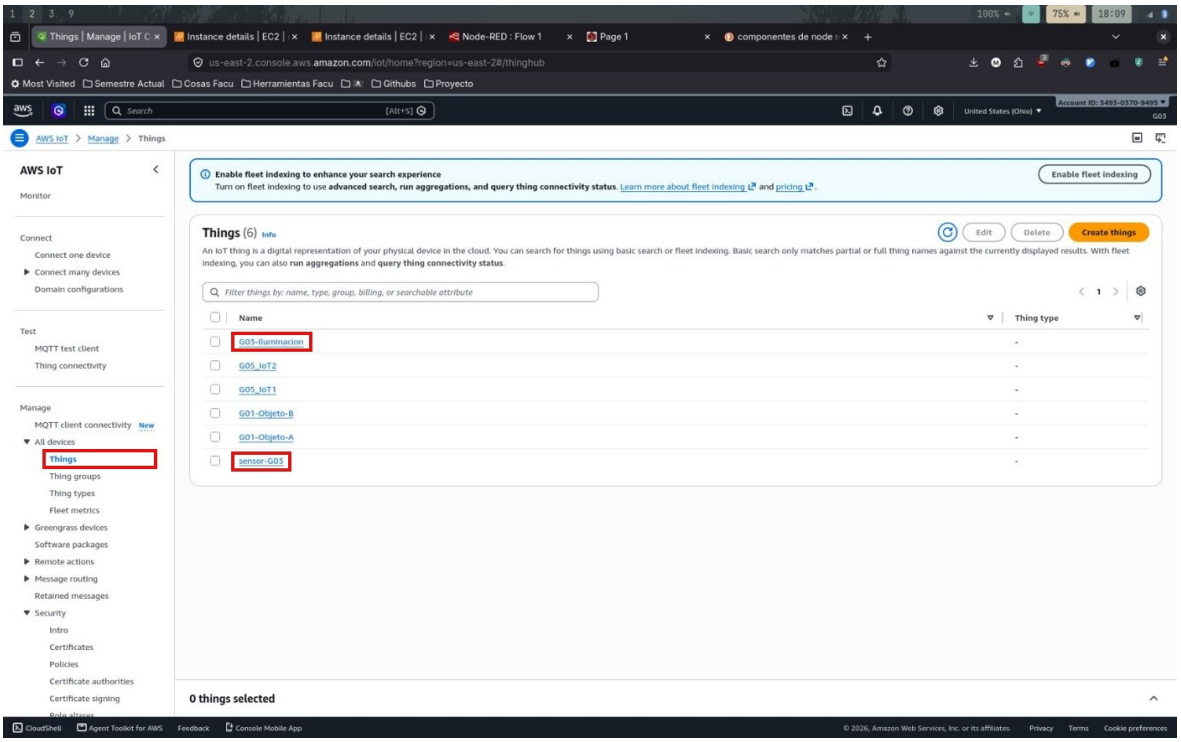


Figura 3: objetos del grupo registrados en AWS IoT: G03-Iluminacion y sensor-G03.

Política del dispositivo

En **Security** → **Policies** editamos la política `sensor-G03-Policy` y guardamos una **versión 2** con efecto **Allow**, acción ***** y recurso *****, para que el dispositivo pueda publicar y suscribirse a los topics que usan nuestras aplicaciones.

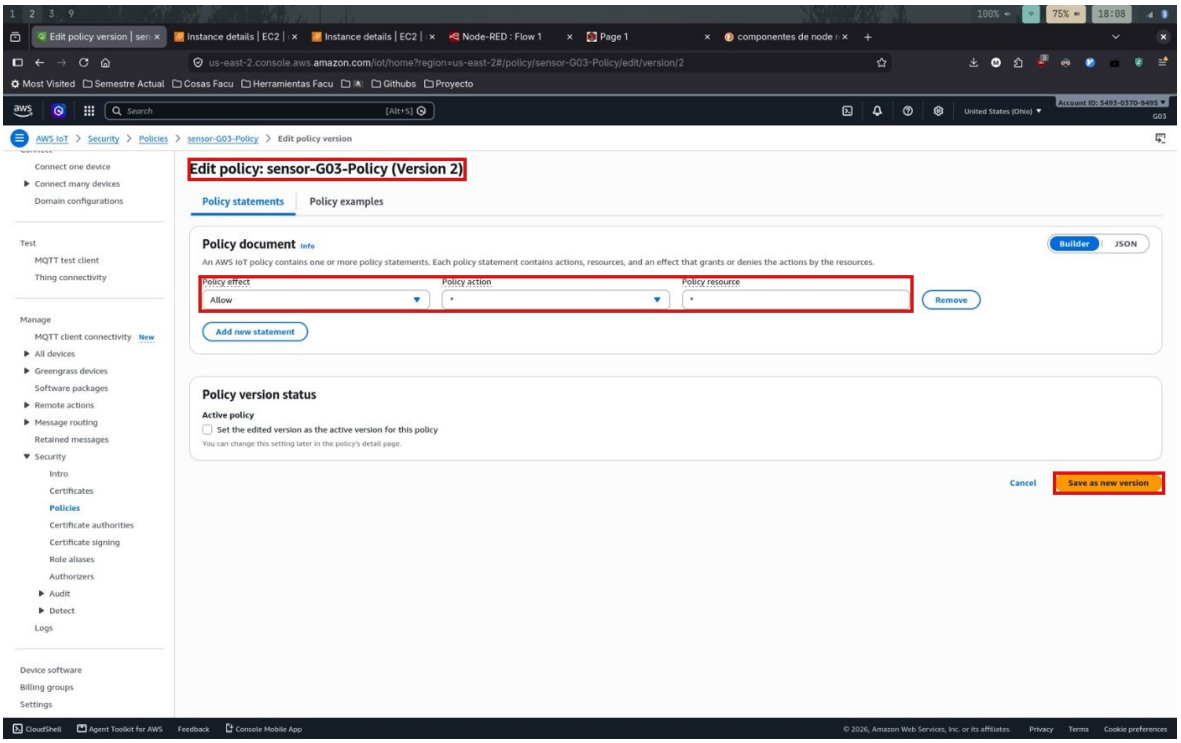


*Figura 4: versión 2 de sensor-G03-Policy: Allow / * / *.*

Bróker MQTT en Node-RED

En Node-RED configuramos el bróker de AWS IoT, al que llamamos `Sensor`, y lo seleccionamos como **Server** en los nodos MQTT del flujo.

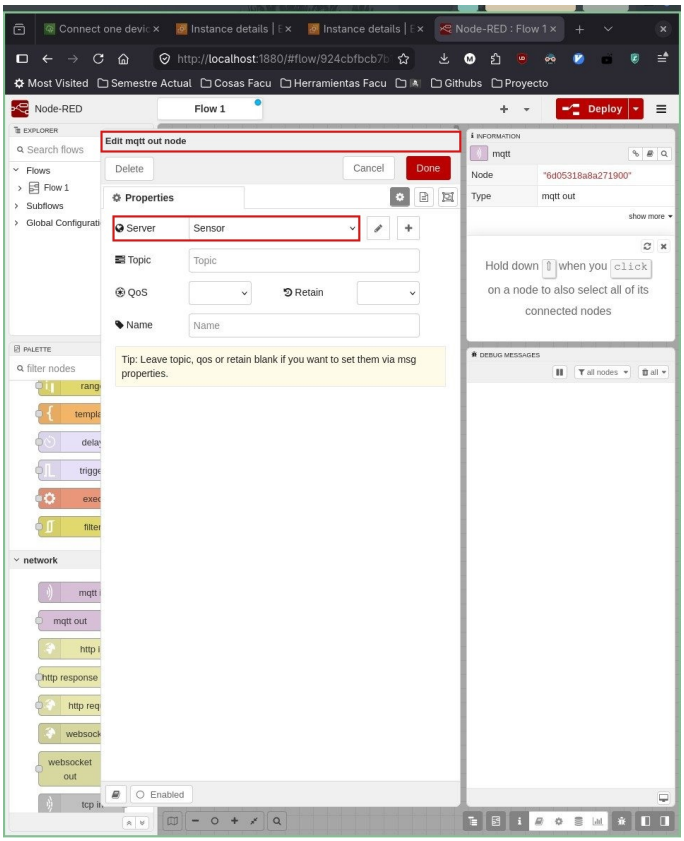


Figura 5: nodo mqtt out con el bróker Sensor seleccionado.

3) Instancia A – Control remoto

El flujo de la Instancia A está formado por los siguientes nodos:

- **ON** y **OFF**: nodos *button* del dashboard. Al presionarlos envían `payload = true` con `topic` `on` y `off` respectivamente.
- **TIMER 5s**: nodo *switch* que envía `true`/`false` con topic `timer`.
- **function 1**: recibe las tres entradas y genera el mensaje `{ led, timer }`. Su primera salida va al nodo **debug 2**, al nodo **mqtt out** y a un nodo **change** que muestra el estado en el dashboard; la segunda salida va al nodo **TIMER**.
- **TIMER**: nodo *delay* cuya salida vuelve a entrar a **function 1**, lo que produce el apagado automático de la luminaria cuando el timer está activo.
- **G03-Control**: nodo *mqtt out* que publica el estado en el topic `G03-Control`.

En el panel de debug se ve el funcionamiento del timer: a las 17:33:22 se publica `{led: 1, timer: true}` y cinco segundos después, a las 17:33:27, `{led: 0, timer: true}`, sin que se presione OFF. En esta primera prueba el topic se llamaba `G03-test`; después lo renombramos a `G03-Control`.

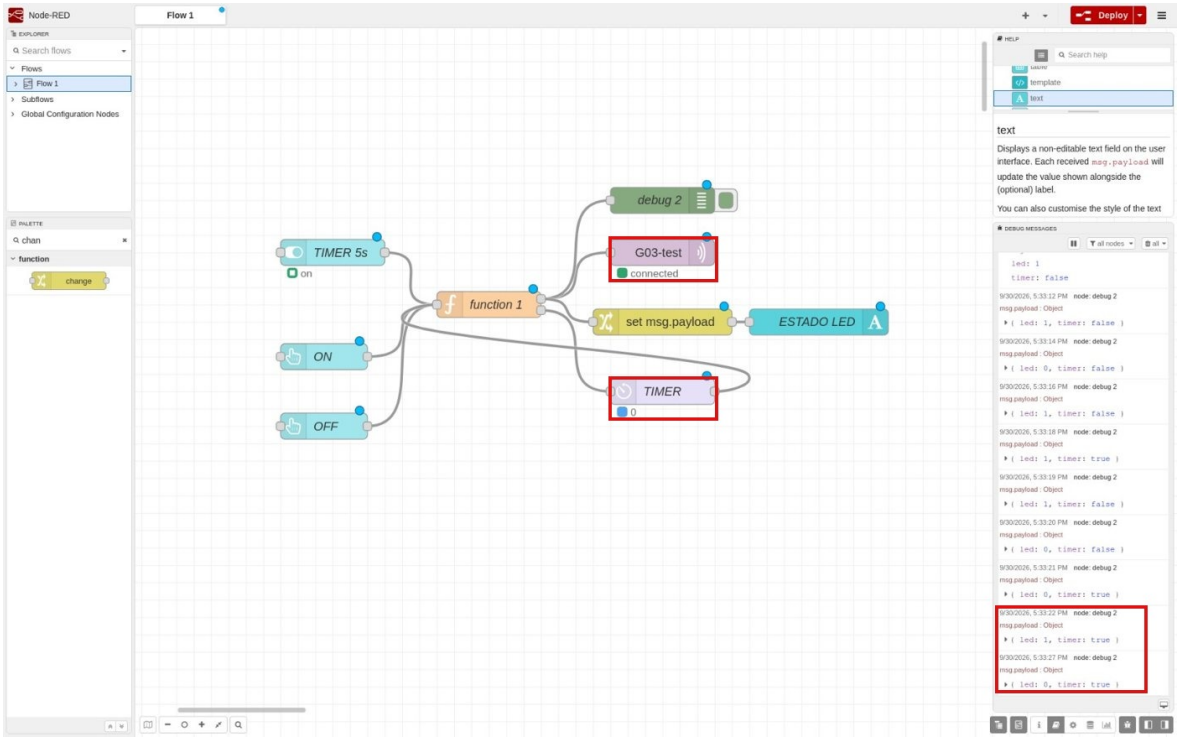


Figura 6: flujo de la Instancia A con debug: encendido con timer y apagado automático 5 s después.

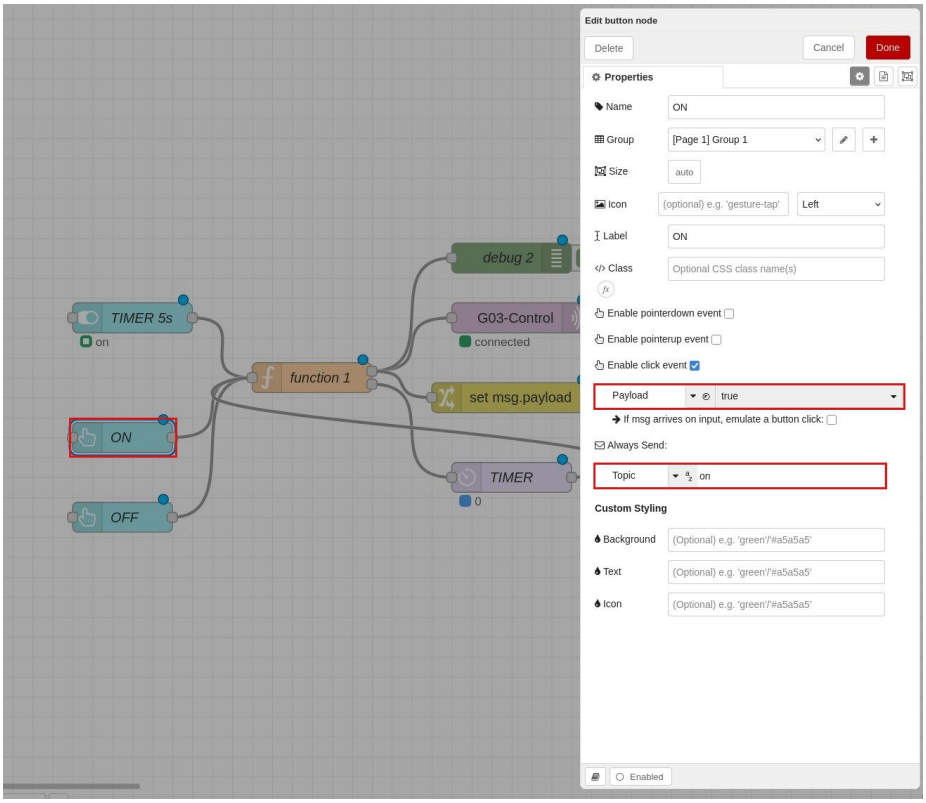


Figura 7: botón ON: payload true y topic “on”.

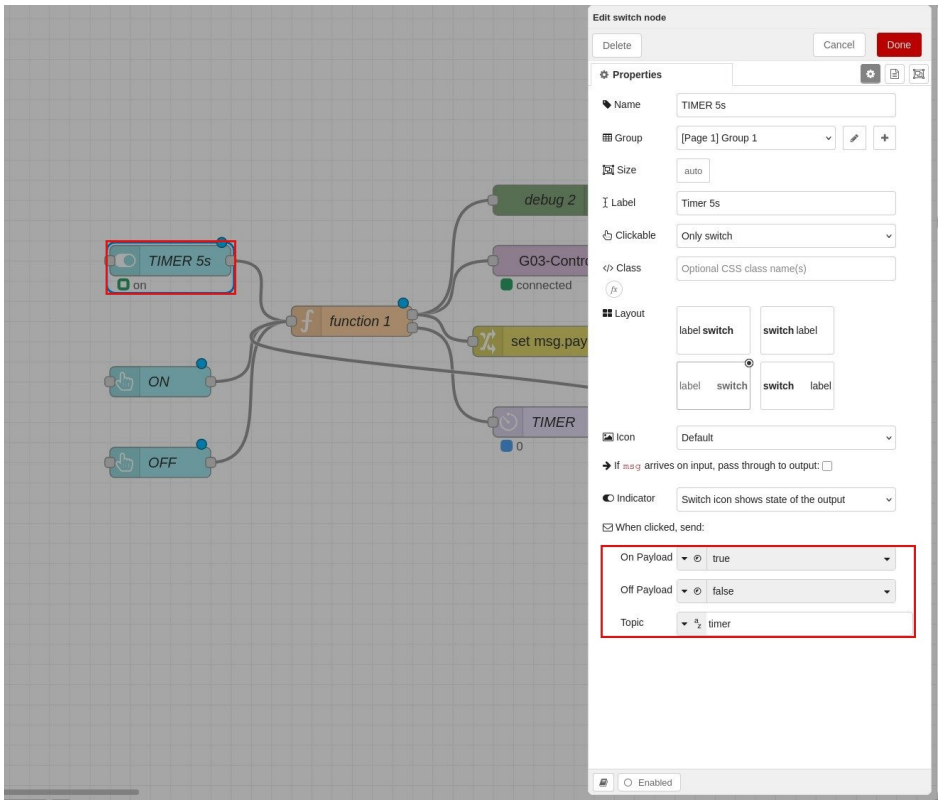


Figura 8: switch TIMER 5s: true/false con topic “timer”.

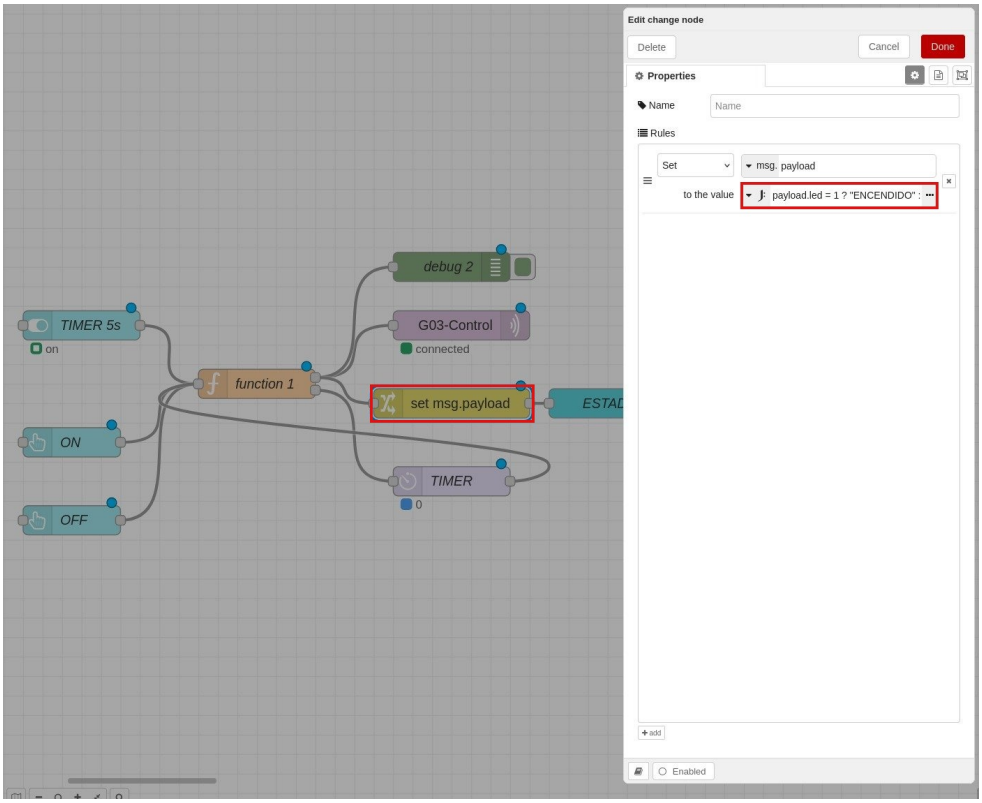


Figura 9: nodo change: traduce payload.led a “ENCENDIDO”/“APAGADO” para el dashboard.

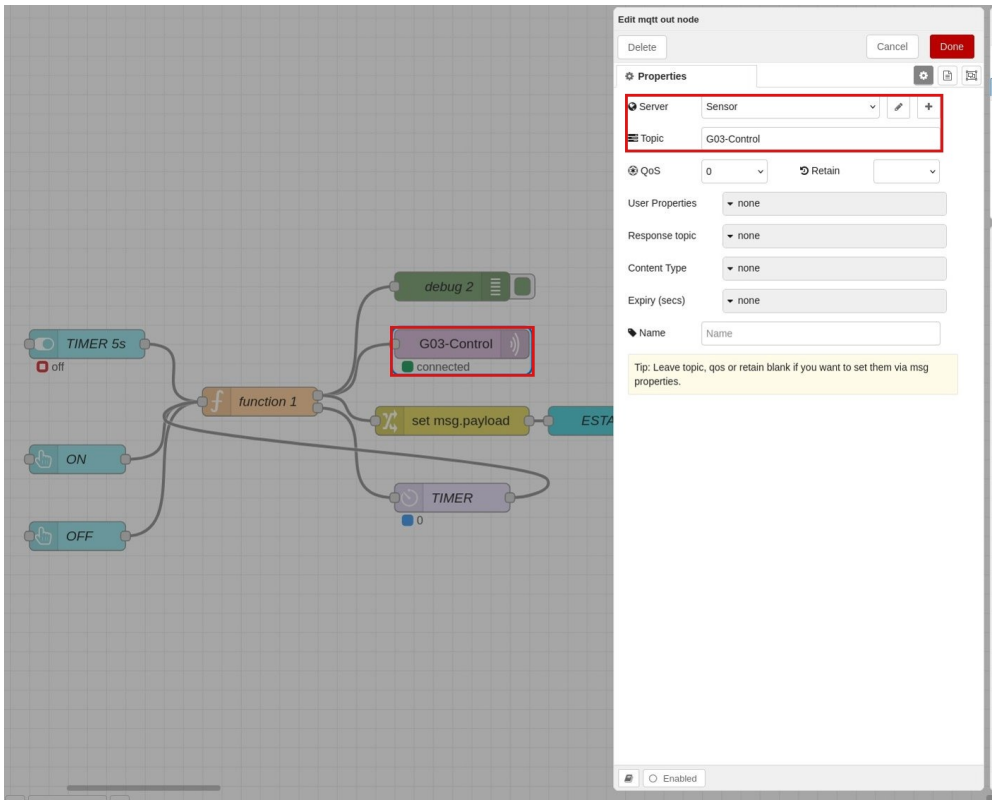


Figura 10: mqtt out G03-Control conectado al bróker.

El dashboard resultante muestra los botones OFF y ON, el switch *Timer 5s* y el **ESTADO LED**:

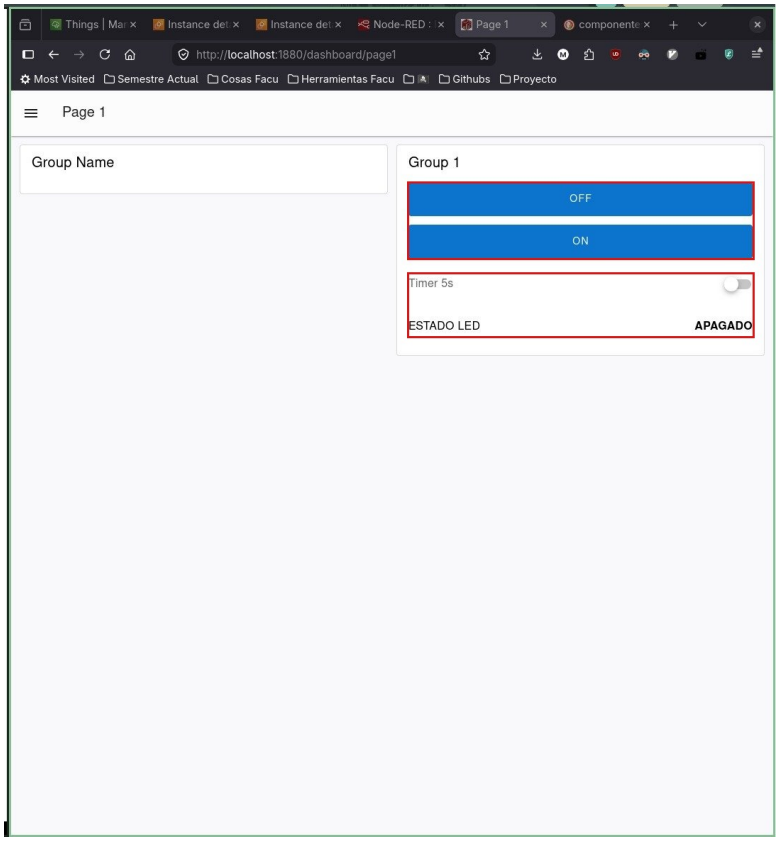


Figura 11: dashboard de la Instancia A (control remoto).

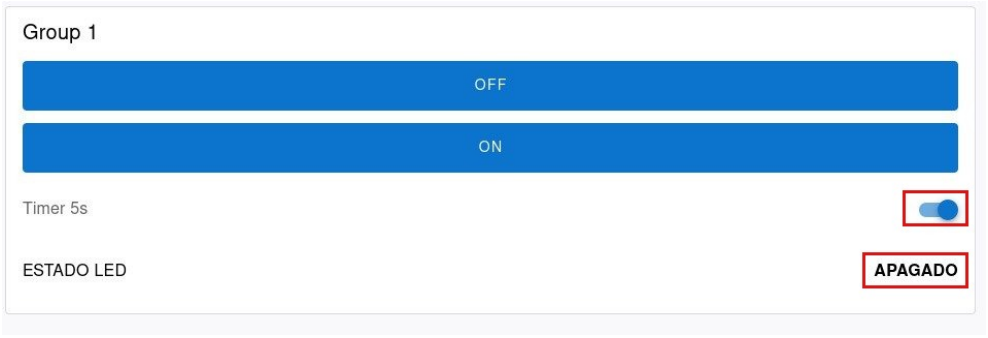


Figura 12: Timer 5s activado desde el control remoto.

4) Instancia B – Iluminación

La Instancia B recibe por MQTT los mensajes publicados por la Instancia A y los muestra en su dashboard mediante tres elementos:

- **ESTADO LED**: texto ENCENDIDO / APAGADO.
- **gauge** (0 a 1, unidad PULSO): simula la luminaria; en 1 el arco se pinta de rojo.
- **ESTADO TIMER**: texto ACTIVADO / DESACTIVADO.

5) Comunicación por MQTT – pruebas de funcionamiento

Con los dos flujos desplegados probamos los cuatro casos de la consigna, con el control remoto (Instancia A) a la izquierda y la luminaria (Instancia B) a la derecha. Las imágenes son fotogramas del video de entrega.

a) Estado inicial: APAGADO

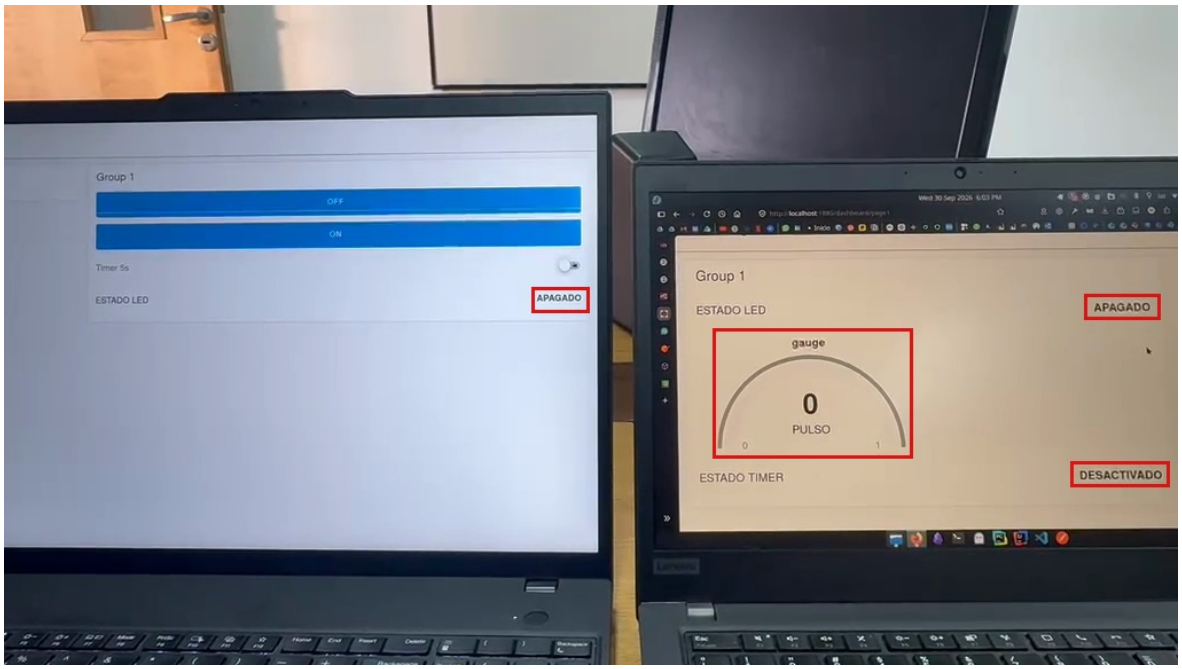


Figura 13: estado inicial: LED apagado, gauge en 0 y timer desactivado.

b) Operación de ENCENDIDO

Al presionar ON en la Instancia A, la Instancia B pasa a ENCENDIDO con el gauge en 1.

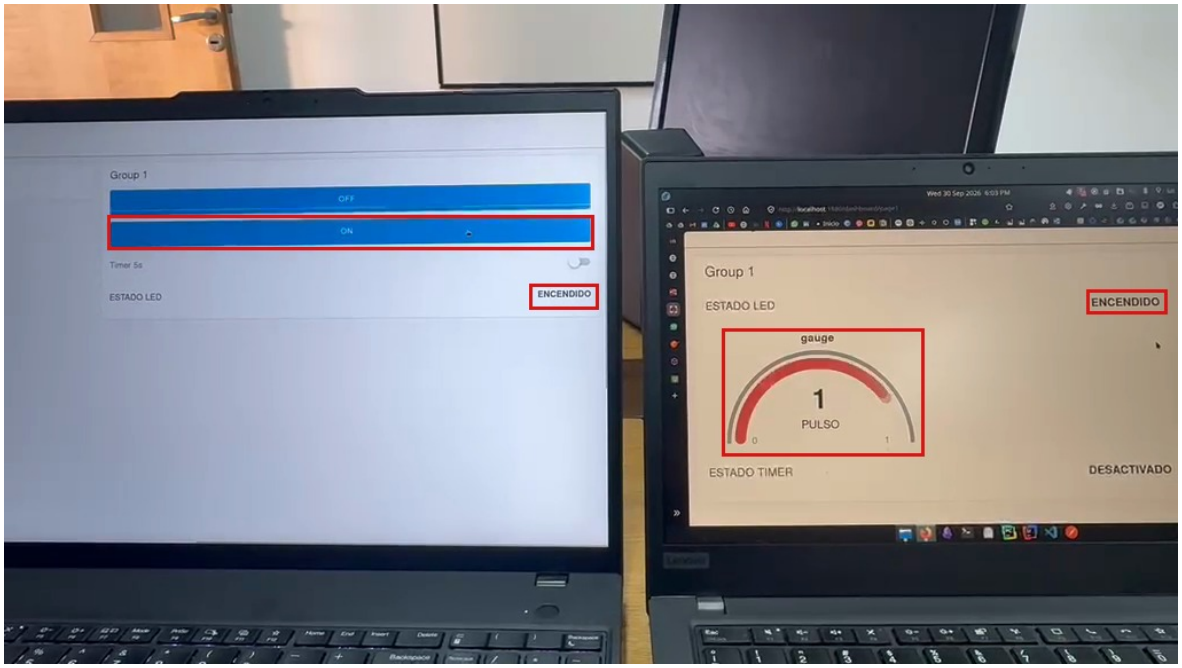


Figura 14: encendido: ambas instancias muestran ENCENDIDO.

c) Activación del TIMER

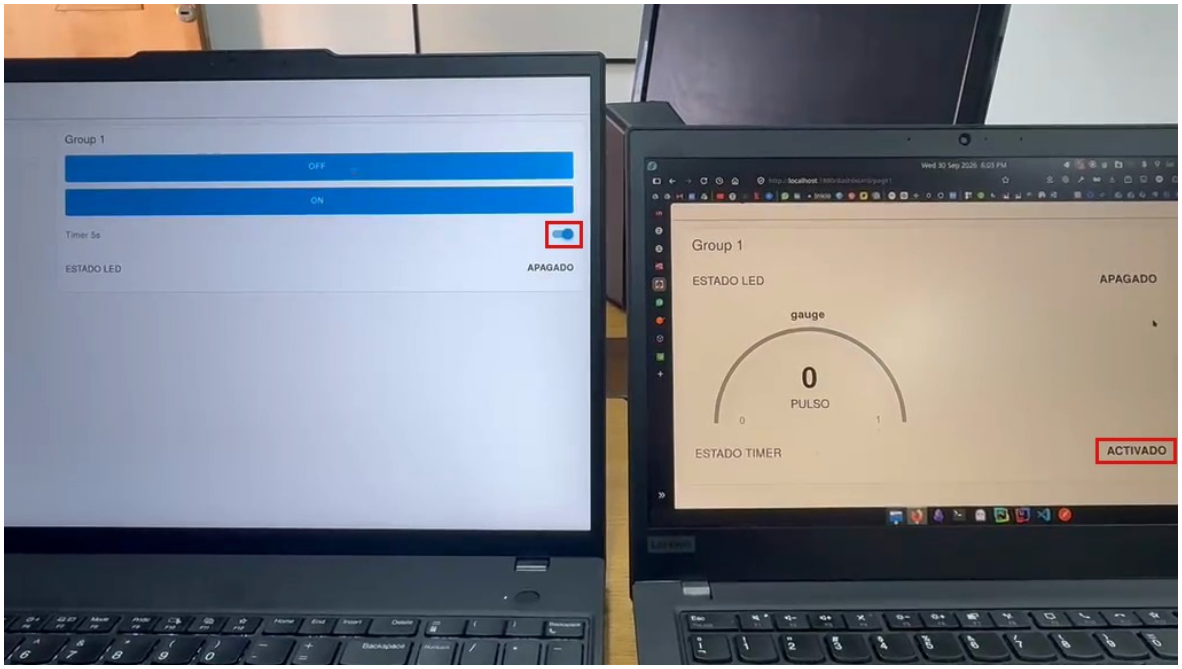


Figura 15: al activar el switch en A, la Instancia B informa ESTADO TIMER: ACTIVADO.

d) ENCENDIDO con TIMER

Con el timer activo presionamos ON: la luminaria se enciende y, pasados 5 segundos, se apaga sola en ambas instancias, mientras el timer sigue ACTIVADO.

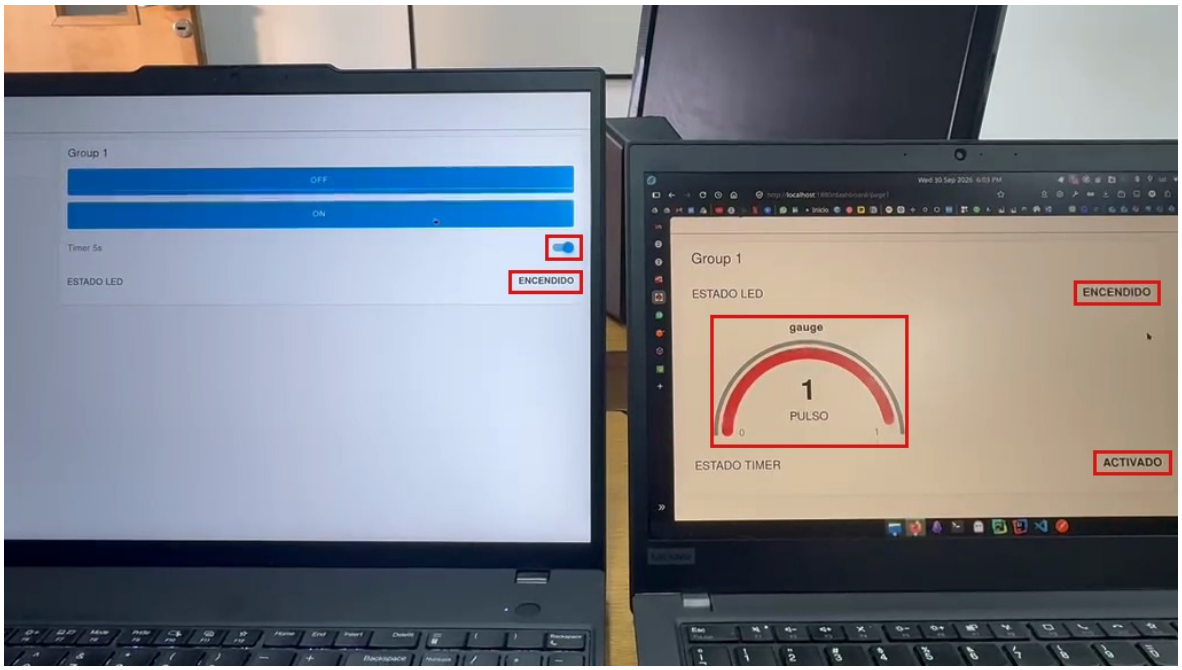


Figura 16: encendido con timer activo.

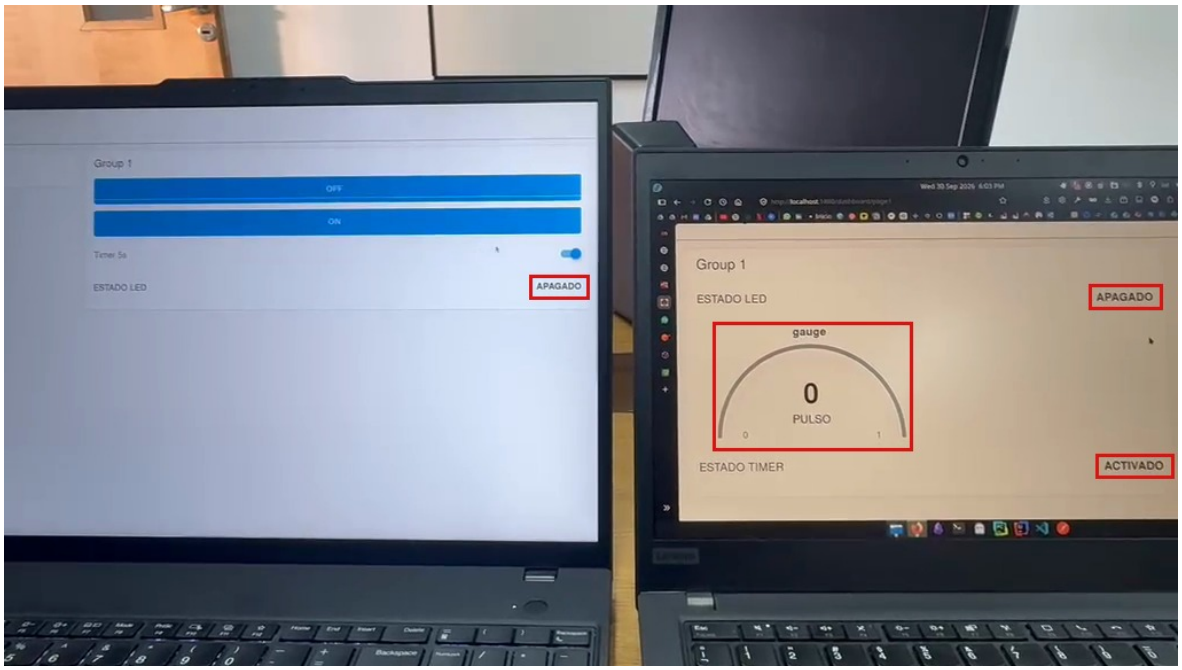


Figura 17: 5 segundos después: apagado automático en ambas instancias.

Conclusiones

- AWS IoT funciona como bróker MQTT: las instancias no se comunican directamente entre sí, sino que una publica y la otra recibe a través de un topic común, lo que desacopla las dos aplicaciones.
- Para poder usar nuestro propio topic fue necesario editar la política del dispositivo en AWS IoT.
- La lógica del control (incluido el timer) está en la Instancia A, y la Instancia B solo refleja el estado recibido, por lo que los dos dashboards siempre muestran lo mismo.
- Los nodos *debug* permitieron verificar el contenido y el momento de cada mensaje; en ellos se comprueba que el apagado automático ocurre 5 s después del encendido.